Software Requirements Specification

For

SnapPlate

Version 1.0 approved

Prepared by Seungwoon Shin, Jihoon Seo, Sangjin Lee, Dongseok Jee

KAIST CS350 Project Team 6

2026.04.07

Table of Contents

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for SnapPlate mobile application version 1.0.
It defines the functional and nonfunctional requirements of the system in accordance with the IEEE 830 standard format and serves as a baseline agreement for requirements among the development team (frontend, backend, and algorithm engineers) and project stakeholders.

SnapPlate is a mobile application that enables users to record food experiences through photos and diary entries, and provides personalized restaurant recommendations based on user data analysis.
This SRS comprehensively describes the requirements of the entire system, including the mobile client, backend system, and analysis and recommendation components.

1.2 Document Conventions

This document follows the conventions below.

DC-1 Requirement Identifiers
Each requirement shall have a unique identifier. Functional requirements shall follow the format “REQ-[Section Number]-[Sequence]” (e.g., REQ-4.1-001). Nonfunctional requirements shall use category prefixes (e.g., PERF-001, SEC-001, QA-001). Design constraints shall use the format “CONST-[Sequence]”.

DC-2 Priority Levels
System features shall be classified into three priority levels: High, Medium, and Low. When necessary, additional attributes such as Benefit, Risk, Cost, and Penalty may be specified using a numerical scale.

DC-3 Terminology
All technical terms and abbreviations used in this document shall be defined in Appendix A (Glossary). Terms shall be introduced with both full names and abbreviations at first occurrence, and thereafter may be used in abbreviated or simplified form.

DC-4 Requirement Statements
All mandatory requirements shall be expressed using “shall”. Recommendations shall be expressed using “should”.

DC-5 Terminology Consistency
The same terminology shall be used consistently throughout the document to refer to the same concept.

1.3 Intended Audience and Reading Suggestions

This document is intended for multiple stakeholders, and the recommended reading order for each audience is as follows.

IA-1 Frontend Developers
Section 2 (Overall Description) → Section 3 (User Interface and Client Interfaces) → Section 4 (User interaction-related features such as camera, upload, diary, and exploration) → Section 5 (Performance and UX-related nonfunctional requirements) → Appendix A (Glossary)

IA-2 Backend Developers
Section 2 → Section 3 (Software and Communication Interfaces) → Section 4 (Data processing, diary management, and restaurant-related features) → Section 5 (Performance, security, and data handling requirements) → Appendix A

IA-3 Algorithm Engineers
Section 2 → Section 4 (Taste analysis and recommendation system features) → Section 5 (Performance and quality-related requirements) → Appendix A

IA-4 Project Managers and Stakeholders
Section 1 → Section 2 → Section 4 for high-level understanding of system functionality

---

1.4 Product Scope

SnapPlate is a mobile application that allows users to record food experiences and receive personalized restaurant recommendations based on their data.
Unlike traditional restaurant review platforms that rely on aggregated ratings from many users, SnapPlate differentiates itself by analyzing individual users' food diary data to provide personalized recommendations.

The scope of the system includes the following components:

(1) Mobile Client
The system shall provide features including restaurant exploration, search and filtering, food diary creation and management, taste analysis visualization, and settings management.
The system shall also support in-app camera and photo upload functionality for recording food experiences.

(2) Backend System
The system shall provide user account management, food diary CRUD operations, image management, restaurant data collection and search, bookmarking functionality, and triggers for taste analysis.

(3) Analysis and Recommendation
The system shall analyze user food diary data to derive user preferences and generate personalized restaurant recommendations.

The following items are out of scope for version 1.0:

- Administrator web dashboard
- Payment and subscription systems
- Social features (sharing or publishing diaries)
- Real-time chat or messaging
- Web or tablet-specific clients

These features may be considered for future expansion but are not included in the current version.

1.5 References

REF-1 IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
REF-2 Expo Documentation (https://docs.expo.dev/)
REF-3 React Native Documentation (https://reactnative.dev/)
REF-4 Supabase Documentation (https://supabase.com/docs)
REF-5 Kakao Local REST API Guide (https://developers.kakao.com/docs/latest/ko/local/dev-guide)
REF-6 FastAPI Documentation (https://fastapi.tiangolo.com/)
REF-7 Apple Human Interface Guidelines (https://developer.apple.com/design/human-interface-guidelines/)
REF-8 Material Design Guidelines (https://m3.material.io/)

---

2. Overall Description

2.1 Product Perspective

SnapPlate is a standalone mobile application that enables users to record food experiences and receive personalized restaurant recommendations.
Unlike traditional systems that rely on aggregated ratings from multiple users, this system focuses on analyzing individual user data to provide personalized recommendations.

The system is designed as a self-contained application and is not intended to replace or extend any existing system.

The system consists of the following major components:

- Mobile Client: Provides user interface for restaurant exploration, diary creation, and analysis visualization
- Backend Server: Handles data processing, authentication, and integration with external APIs
- Data Storage: Stores user data and images
- External Services: Includes restaurant data APIs, map services, authentication services, and AI analysis services

The system relies on external services for certain functionalities, particularly restaurant data retrieval and map integration.

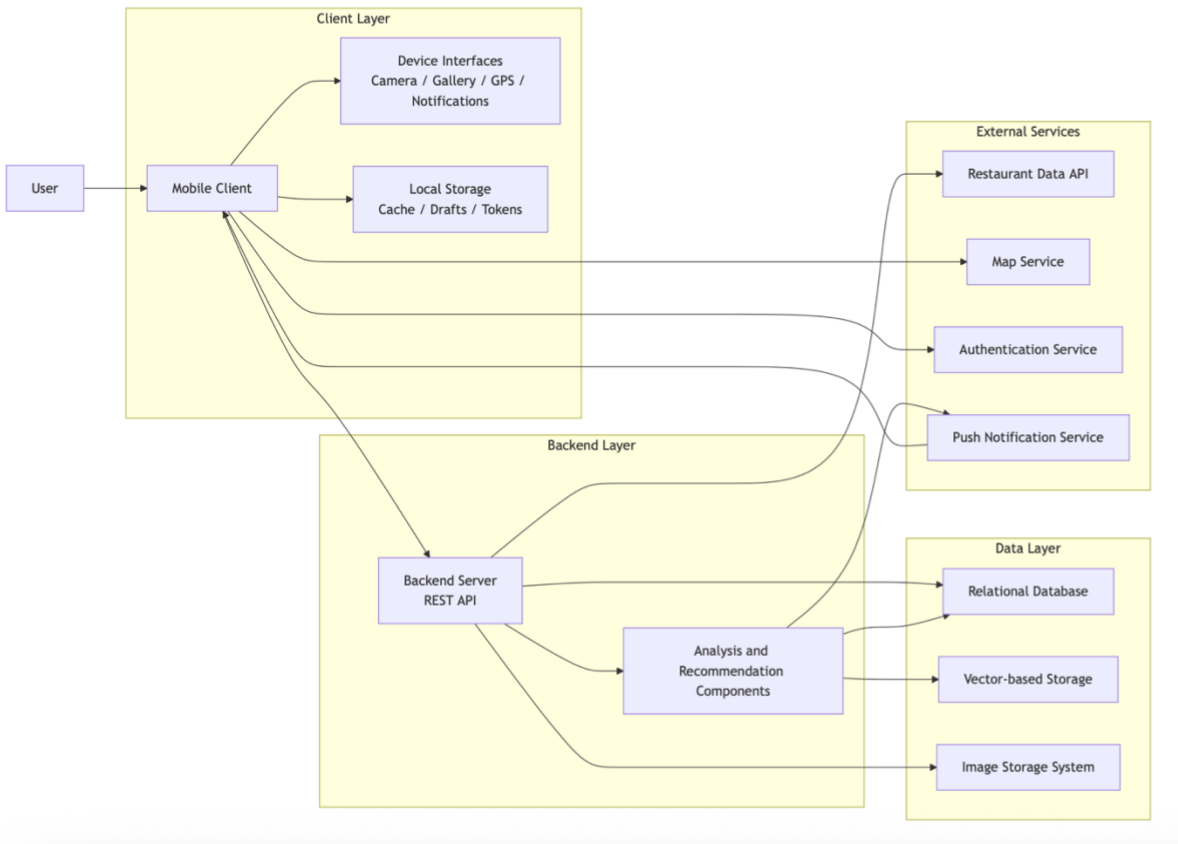

Figure 1. System Architecture Diagram

2.2 Product Functions

The system shall provide the following major functions:

PF-1 Restaurant Exploration and Recommendation
The system shall retrieve nearby restaurants based on the user's location and provide personalized recommendations.

PF-2 Restaurant Search and Filtering
The system shall allow users to search for restaurants using keywords and filter results based on conditions.

PF-3 Bookmark Management
The system shall allow users to save and manage preferred restaurants.

PF-4 Food Diary Creation
The system shall allow users to record food experiences using photos, ratings, and comments.

PF-5 Food Diary Management
The system shall allow users to view, edit, and delete diary entries.

PF-6 Taste Analysis
The system shall analyze user food diary data and present preference insights in a visualized format.

PF-7 User Account Management
The system shall provide login, profile management, and account deletion functionality.

PF-8 Settings Management
The system shall provide application and user preference settings.

PF-9 Restaurant Data Management
The system shall collect and update restaurant data using external APIs.

Detailed requirements for these functions are defined in Section 4.

2.3 User Classes and Characteristics

UC-1 General User (Primary User)

The general user is the primary user of the application, utilizing food diary and restaurant recommendation features.
The target audience consists of mobile-savvy users in their 20s and 30s who seek to record dining experiences and receive personalized recommendations.

This user group has intermediate or higher digital literacy and is familiar with mobile applications, map services, and social platforms.
They prefer intuitive and fast interfaces rather than complex configurations or lengthy input processes.

Primary usage goals include:

- Recording and managing food experiences
- Viewing personalized restaurant recommendations
- Discovering new restaurants

The system shall support usage in mobile environments, often during short interaction periods such as immediately after meals, and shall provide fast response times and minimal input requirements.

UC-2 System Administrator (Secondary User)

The system administrator is responsible for maintaining system operation, including server and database management.
This user does not directly interact with the mobile application interface and typically operates in a development or operational environment.

This user group has high technical expertise and requires knowledge of servers, databases, and APIs.

Primary responsibilities include:

- System operation and maintenance
- Data management and issue handling
- External service integration management

Administrative tools are outside the scope of this application.

2.4 Operating Environment

The system shall operate as a mobile application on iOS and Android platforms.

The client shall run on mobile operating systems (iOS 16 or later, Android 10 or later) and requires network connectivity (Wi-Fi or cellular).

The backend shall operate as a REST API server using HTTPS communication.

Data shall be stored in a relational database, and images shall be stored in a separate storage system.

The system depends on external services (APIs, authentication services, and analysis services), and some functionality may be affected by their availability.

2.5 Design and Implementation Constraints

The system shall comply with the following constraints:

- The system shall operate in a mobile environment and support both iOS and Android platforms
- The client shall be implemented using a single codebase
- The system shall depend on external APIs (restaurant data, map services, authentication services)
- External API policies and limitations shall be respected

- Sensitive information (e.g., API keys) shall be securely managed
- All communication shall be conducted over HTTPS
- The application shall comply with app store policies

2.6 User Documentation

The system shall provide the following user documentation:

- Onboarding guide
- Feature explanation tooltips
- Terms of service and privacy policy
- Customer support contact information

Documentation shall be accessible within the application.

2.7 Assumptions and Dependencies

The system assumes the following conditions:

- Users have network connectivity
- User devices provide location data (GPS)
- External APIs (restaurant data, map services) are available
- Authentication and database services are operational
- AI analysis services are available

Changes or failures in these external dependencies may affect system functionality.

---

3. External Interface Requirements

3.1 User Interfaces

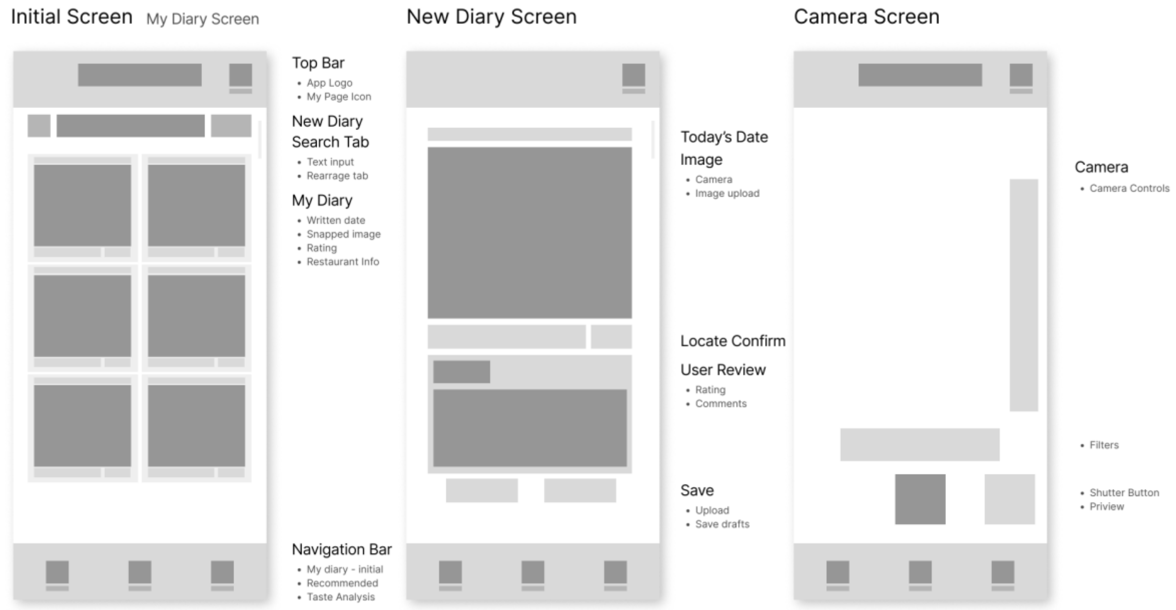

Figure 2. UI Wireframe 1

Figure 3. UI Wireframe 2

3.1.1 UI Design Principles

REQ-UI-001: The system shall comply with Apple Human Interface Guidelines for iOS.
REQ-UI-002: The system shall comply with Material Design guidelines for Android.

REQ-UI-003: The system shall comply with WCAG 2.1 AA accessibility standards.
REQ-UI-004: The minimum touch target size shall be 44×44 dp or larger.
REQ-UI-005: The system shall respect system font scaling and user accessibility settings.

3.1.2 User Feedback and Messaging

REQ-UI-006: The system shall provide visual feedback for successful actions.
REQ-UI-007: The system shall display warning messages for user actions requiring attention.
REQ-UI-008: The system shall display error messages with recovery or retry options.
REQ-UI-009: The system shall guide users to system settings when required permissions are denied.
REQ-UI-010: The system shall notify users when background processing (e.g., analysis) is completed.

3.1.3 Screen Composition

REQ-UI-011: The restaurant exploration screen shall include a search bar, recommendation section, and restaurant list.
REQ-UI-012: The recommendation section shall be presented in a horizontally scrollable format.
REQ-UI-013: The restaurant list shall be displayed as a vertical list.
REQ-UI-014: The diary list screen shall support infinite scrolling.
REQ-UI-015: Each diary item shall include image, restaurant name, rating, date, and summary.
REQ-UI-016: The diary detail screen shall include image, rating, comment, and location information.

REQ-UI-017: The bookmark screen shall provide a searchable list of saved restaurants.
REQ-UI-018: The system shall display an appropriate empty state when no data is available.

REQ-UI-019: The settings screen shall be structured as grouped options.
REQ-UI-020: The settings shall include notification, account, and application preferences.

3.1.4 Common UI Elements

REQ-UI-021: The system shall display loading states using placeholder UI.
REQ-UI-022: The system shall display dialogs using modal components.
REQ-UI-023: The system shall support bottom-aligned modal panels.

3.2 Hardware Interfaces

REQ-HW-001: The system shall use the device camera for capturing food images.
REQ-HW-002: The system shall use GPS to obtain user location.
REQ-HW-003: The system shall require network connectivity (Wi-Fi or cellular).
REQ-HW-004: The system shall require sufficient local storage for image data.
REQ-HW-005: The system shall support iOS 16 or later and Android 10 or later.

REQ-HW-006: The system shall provide haptic feedback for user interactions.
REQ-HW-007: The system shall detect and respond to network status changes.

3.3 Software Interfaces

3.3.1 Backend and Database

REQ-SW-001: The system shall communicate with backend services using REST APIs.
REQ-SW-002: Data exchange shall use JSON format.
REQ-SW-003: The system shall use token-based authentication.

REQ-SW-004: The system shall store data in a relational database.
REQ-SW-005: The system shall support vector-based data storage for analysis purposes.
REQ-SW-006: The system shall store image data in a dedicated storage system.

3.3.2 External Services

REQ-SW-007: The system shall retrieve restaurant data from external APIs.
REQ-SW-008: The system shall integrate map services for location display.
REQ-SW-009: The system shall support deep linking to external map applications.

REQ-SW-010: The system shall support push notification services.

3.3.3 Device-Level Interfaces

REQ-SW-011: The system shall obtain location data through device APIs.
REQ-SW-012: The system shall handle push notifications through platform APIs.
REQ-SW-013: The system shall securely store authentication tokens.

3.4 Communication Interfaces

REQ-COM-001: All communication shall use HTTPS (TLS 1.2 or higher).
REQ-COM-002: API requests shall use JSON or multipart/form-data formats.
REQ-COM-003: Requests shall include authentication tokens in headers.

REQ-COM-004: Image uploads shall use multipart/form-data.
REQ-COM-005: The system shall implement retry logic for failed requests.

REQ-COM-006: Push notifications shall be delivered through platform services.
REQ-COM-007: The system shall synchronize data based on network availability.

REQ-COM-008: The system shall handle external API failures gracefully.

---

4. System Features

4.1 User Account and Profile Management

4.1.1 Description and Priority

This feature is responsible for user authentication (Supabase Auth), onboarding, profile management, and account lifecycle management.
A user record shall be automatically created upon login, and the system shall manage nickname, profile image, and taste type information.

Priority: High
Benefit: 8 | Penalty: 8 | Cost: 5 | Risk: 6

4.1.2 Stimulus/Response Sequences

The user logs in.
The system verifies the authentication token and retrieves user information.

The user logs in for the first time.
The system creates a user record.

The user updates the profile.
The system stores the updated information.

The user requests logout.
The system removes authentication data.

The user requests account deletion.
The system performs a two-step confirmation process.

4.1.3 Functional Requirements

REQ-4.1-001: The system shall provide login functionality based on Supabase Auth.
REQ-4.1-002: The system shall automatically create a user record if none exists upon login.
REQ-4.1-003: The system shall return nickname, email, profile_image_url, and taste_type.
REQ-4.1-004: The nickname shall be editable up to 30 characters.
REQ-4.1-005: The email shall not be editable.
REQ-4.1-006: The system shall provide profile image upload functionality.
REQ-4.1-007: The system shall delete tokens and cached data upon logout.
REQ-4.1-008: The system shall record deleted_at upon account deletion.
REQ-4.1-009: Account deletion shall include a 30-day grace period.
REQ-4.1-010: The system shall allow cancellation of deletion if the user logs in within the grace period.
REQ-4.1-011: The system shall restrict access to user data to the owner only.

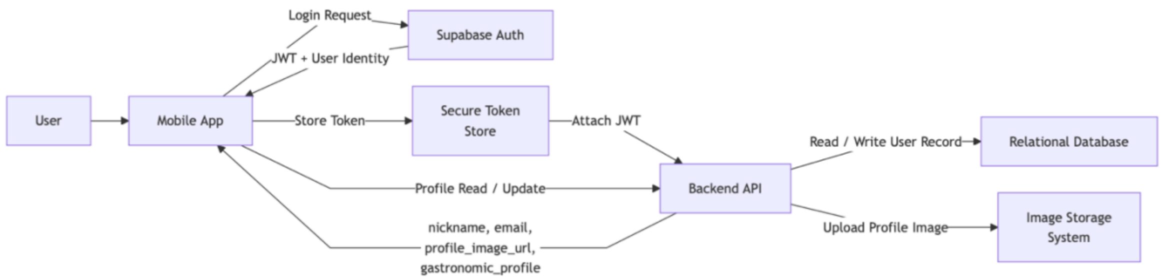

Figure 4. Authentication and Profile Data Flow Diagram

4.2 Restaurant Exploration and Search

4.2.1 Description and Priority

This feature retrieves nearby restaurants based on the user's GPS location and provides search, filtering, and sorting functionality.
The system shall collect and cache data using an external API (Kakao).

Priority: High

4.2.2 Stimulus/Response Sequences

The user enters the exploration tab.
The system checks GPS location and retrieves restaurant data.

The user scrolls.
The system loads additional data.

The user enters a search query.
The system displays results after applying a 300ms debounce.

The user applies filters.
The system displays filtered results.

The user selects a restaurant.
The system navigates to Kakao Map.

4.2.3 Functional Requirements

REQ-4.2-001: The system shall retrieve restaurants within a radius based on GPS location.
REQ-4.2-002: The restaurant list shall be sorted by distance.
REQ-4.2-003: The system shall support cursor-based infinite scrolling.
REQ-4.2-004: The system shall load up to 20 items per request.
REQ-4.2-005: The system shall provide search functionality.
REQ-4.2-006: The system shall apply a 300ms debounce to search input.
REQ-4.2-007: The system shall provide filters for category, distance, and rating.
REQ-4.2-008: The system shall provide sorting options (distance, rating, recommendation).
REQ-4.2-009: The system shall use Kakao Map deep linking as the primary option.
REQ-4.2-010: The system shall fall back to a web URL if the app is not installed.
REQ-4.2-011: The system shall return cached data if the API request fails.
REQ-4.2-012: The system shall display an error message in case of network failure.
REQ-4.2-013: The system shall display a permission request UI if location permission is not granted.

4.3 Restaurant Bookmark Management

4.3.1 Description and Priority

This feature allows users to bookmark and manage restaurants.

Priority: Medium

4.3.2 Stimulus/Response Sequences

The user presses the bookmark button.
The system saves the restaurant.

The user presses it again.
The system removes the bookmark.

The user requests the bookmark list.
The system returns the data.

4.3.3 Functional Requirements

REQ-4.3-001: The user shall be able to bookmark restaurants.
REQ-4.3-002: Duplicate bookmarks for the same restaurant shall not be allowed.
REQ-4.3-003: The system shall return an error for duplicate requests.
REQ-4.3-004: The bookmark list shall be sorted by most recent.
REQ-4.3-005: The system shall provide an unbookmark function.
REQ-4.3-006: The system shall revert state if server synchronization fails.

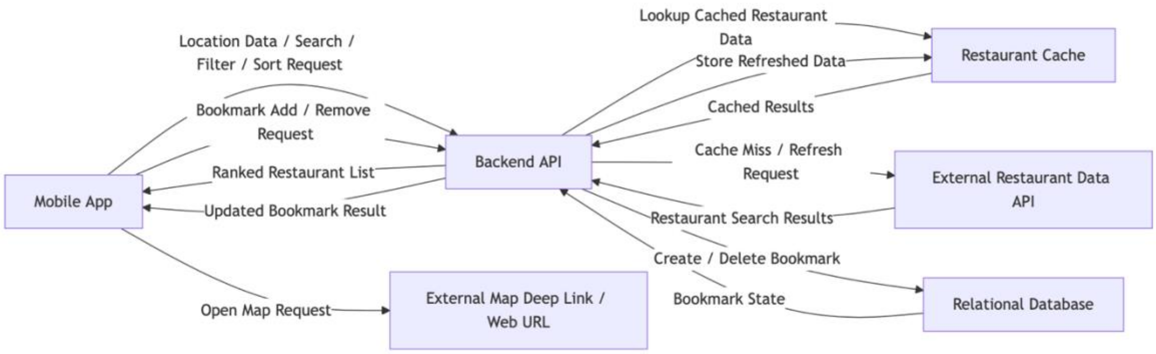

Figure 5. Restaurant Exploration and Bookmark Data Flow Diagram

---

4.4 In-App Camera

4.4.1 Description and Priority

This feature provides an in-app camera for capturing food images, including capture, preview, retake, filters, and metadata extraction.

Priority: High

4.4.2 Stimulus/Response Sequences

The user launches the camera.
The system requests permission.

The user captures an image.
The system displays a preview screen.

The user confirms.
The system transfers the data.

4.4.3 Functional Requirements

REQ-4.4-001: The system shall request camera and location permissions.
REQ-4.4-002: The system shall provide guidance to system settings if permission is denied.
REQ-4.4-003: The system shall use only the rear camera.
REQ-4.4-004: The system shall provide photo capture functionality.
REQ-4.4-005: The system shall provide preview and retake functionality.
REQ-4.4-006: The system shall provide zoom, flash, and filter features.
REQ-4.4-007: The system shall extract EXIF metadata.
REQ-4.4-008: The system shall use GPS if location metadata is unavailable.
REQ-4.4-009: The system shall provide retry functionality in case of errors.

4.5 Photo Upload

4.5.1 Description and Priority

This feature allows users to select and upload images from the device gallery. Multiple image selection is supported within a defined limit.

Priority: High

4.5.2 Stimulus/Response Sequences

The user opens the gallery.
The system requests permission.

The user selects one or more images.
The system uploads the selected images.

4.5.3 Functional Requirements

REQ-4.5-001: The system shall request gallery access permission.
REQ-4.5-002: The system shall allow selection of one or more images.
REQ-4.5-003: The system shall limit the number of selectable images to a maximum of 10.
REQ-4.5-004: The system shall verify EXIF metadata for each selected image.
REQ-4.5-005: The system shall check for location and timestamp information in EXIF metadata.
REQ-4.5-006: The system shall request user input if location or timestamp metadata is missing.
REQ-4.5-007: The upload shall be performed using multipart/form-data.
REQ-4.5-008: The system shall display upload progress.

REQ-4.5-009: The system shall retry up to three times upon failure.

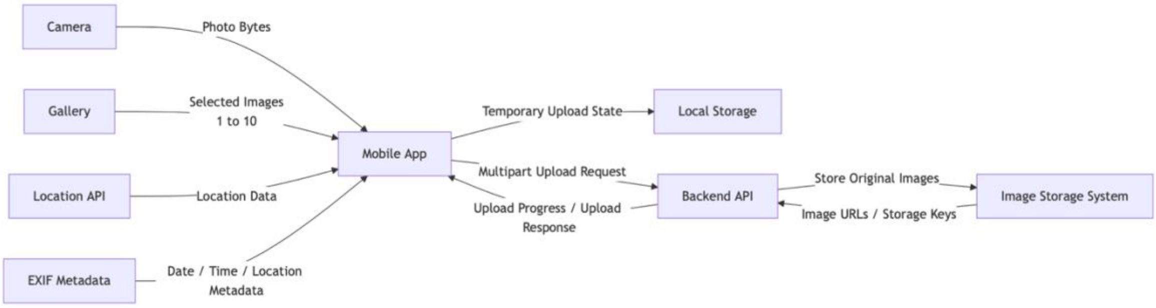

Figure 6. Media Upload Data Flow Diagram

4.6 Food Diary Creation and Management

4.6.1 Description and Priority

This is a core feature that creates and manages food diary entries based on photos, user input (such as ratings and comments), and metadata (such as location and time).

Priority: High

4.6.2 Stimulus/Response Sequences

The user inputs data.
The system automatically populates metadata such as date, time, and location.

The user enters additional information such as rating and comments.

The user saves the entry.
The system validates input data.

If validation succeeds, the system stores the diary entry.
If validation fails, the system displays an error message.

4.6.3 Functional Requirements

REQ-4.6-001: The system shall provide photo-based diary creation functionality.
REQ-4.6-002: The diary entry shall include at least one photo.
REQ-4.6-003: The system shall allow multiple photos per diary entry (up to 10 images).
REQ-4.6-004: The system shall automatically populate date, time, and location based on metadata.
REQ-4.6-005: The user shall be able to modify date, time, and location values.

REQ-4.6-006: The system shall provide input fields for user-generated data, including rating and comments.
REQ-4.6-007: The rating shall be required input.
REQ-4.6-008: The comment field shall be optional.
REQ-4.6-009: The system shall provide automatic restaurant suggestion functionality.
REQ-4.6-010: The user shall be able to manually select or skip restaurant selection.
REQ-4.6-011: The system shall validate all input data before saving.
REQ-4.6-012: The system shall support offline storage and synchronization.

Figure 7. Food Diary Generation and Management Data Flow Diagram

---

4.7 Food Diary Retrieval and Exploration

4.7.1 Description and Priority

This feature allows users to view and explore all stored food diary entries.
Users shall be able to efficiently manage their records through sorting, search, and detailed views, and shall be able to check restaurant locations through map integration in the diary detail view.

Priority: High
Benefit: 8 | Penalty: 7 | Cost: 5 | Risk: 5

4.7.2 Stimulus/Response Sequences

The user enters the “My Diary” tab.
The system retrieves and displays stored food diary entries in descending order by recency.

The user scrolls down.
The system loads additional data incrementally and appends it to the list.

The user changes the sorting option.
The system reorders the list based on the selected criteria.

The user enters a search query.
The system filters the diary list in real time based on the keyword.

The user selects a specific diary entry.
The system displays the diary detail screen.

The user selects “View on Map” from the detail screen.
The system navigates to the Kakao Map application or web page.

If no diary entries exist:
The system displays an empty state UI with a guidance message.

4.7.3 Functional Requirements

REQ-4.7-001: The system shall allow retrieval of stored food diary entries in list format.
REQ-4.7-002: The system shall use recency as the default sorting criterion.
REQ-4.7-003: The system shall support cursor-based infinite scrolling.
REQ-4.7-004: The system shall use a virtualized list to handle large datasets efficiently.
REQ-4.7-005: The system shall allow sorting by multiple criteria (recency, rating, distance, etc.).
REQ-4.7-006: The system shall provide search functionality based on comment, restaurant name, and location.
REQ-4.7-007: The system shall update search results in real time.
REQ-4.7-008: The system shall display photo, rating, comment, date, location, and restaurant information in the detail view.
REQ-4.7-009: The system shall navigate to Kakao Map via the “View on Map” feature.
REQ-4.7-010: The system shall fall back to a web page if the map application is not installed.
REQ-4.7-011: The system shall display an empty state UI when no data exists.
REQ-4.7-012: The system shall display an error message in case of network failure.

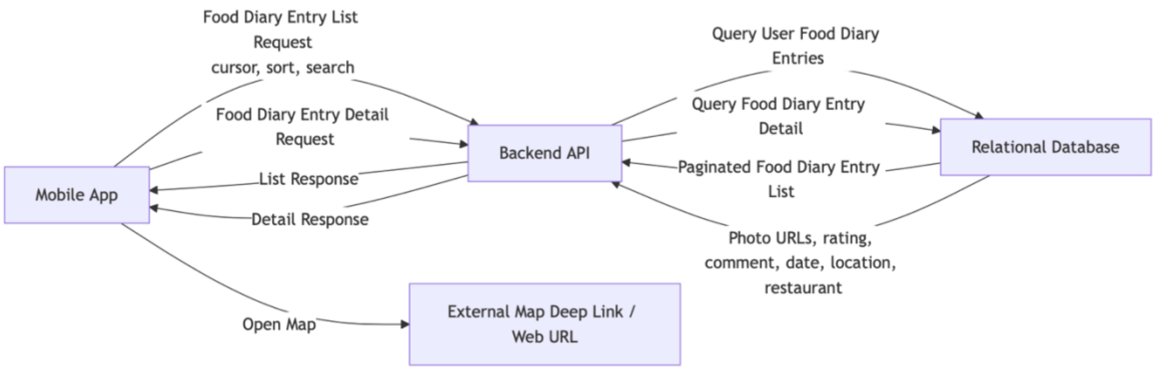

Figure 8. Diary Retrieval Data Flow Diagram

4.8 Taste Analysis and Profile Generation

4.8.1 Description and Priority

This feature analyzes user food diary data to derive individual food preferences and presents the results in a visually intuitive format.
Users shall be able to understand their food preferences, consumption patterns, and gastronomic profile.

Priority: High

4.8.2 Stimulus/Response Sequences

When a sufficient number of food diary entries are accumulated:
The system performs taste analysis in the background.

When analysis is completed:
The system stores the result data.

The user enters the “Taste Analysis” screen.
The system retrieves and displays the latest analysis results.

The user reviews the results.
The system presents information through various visualization elements.

The user adds a new diary entry.
The system updates the analysis results.

If data is insufficient:
The system displays a message indicating that analysis is not available.

4.8.3 Functional Requirements

REQ-4.8-001: The system shall perform taste analysis based on user food diary data.
REQ-4.8-002: The system shall perform analysis only when a minimum number of entries is available.
REQ-4.8-003: The system shall calculate preference scores based on food categories, ratings, and visit frequency.
REQ-4.8-004: The system shall derive a user gastronomic profile (e.g., type classification or label).
REQ-4.8-005: The system shall calculate the top N frequently consumed food categories.
REQ-4.8-006: The system shall calculate average rating and distribution statistics.
REQ-4.8-007: The system shall analyze time-of-day patterns (e.g., breakfast, lunch, dinner).
REQ-4.8-008: The system shall present analysis results visually.
REQ-4.8-009: The system shall present category preferences as graphs (e.g., bar chart or pie chart).
REQ-4.8-010: The system shall present gastronomic profile information as cards or descriptive text.
REQ-4.8-011: The system shall provide summary statistics (e.g., average rating, total entries).
REQ-4.8-012: The system shall provide user-friendly explanatory text.
REQ-4.8-013: The system shall periodically update analysis results.
REQ-4.8-014: The system shall provide user notifications upon analysis completion.
REQ-4.8-015: The system shall display a message when data is insufficient for analysis.
REQ-4.8-016: The system shall perform analysis asynchronously without affecting user

experience.

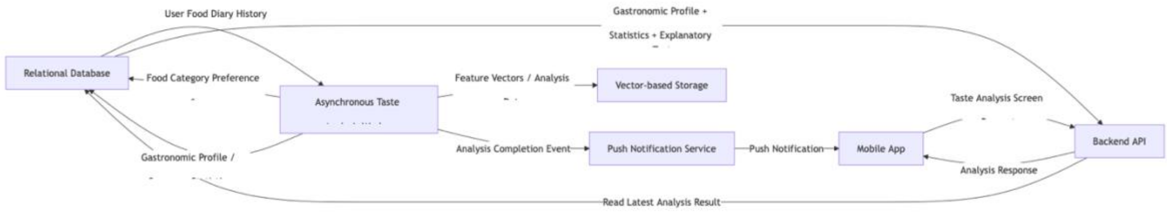

Figure 9. Taste Analysis Data Flow Diagram (For details, refer to Appendix B.)

4.9 Restaurant Profiling and Recommendation System

4.9.1 Description and Priority

This feature provides personalized restaurant recommendations based on user preference data and restaurant data.
The system shall use a hybrid recommendation approach combining content-based filtering, collaborative filtering, and context-based filtering.

Priority: High
Benefit: 9 | Penalty: 9 | Cost: 8 | Risk: 8

4.9.2 Stimulus/Response Sequences

The user requests the recommendation feed.
The system retrieves user preference data.
The system generates a candidate set of restaurants.
The system calculates scores for each candidate.
The system sorts the recommendation results.
The system displays the results to the user.

If data is insufficient:
The system displays a message indicating insufficient recommendations.

4.9.3 Functional Requirements

REQ-4.9-001: The system shall generate recommendations based on user preference data.
REQ-4.9-002: The system shall generate candidate restaurants based on restaurant data.
REQ-4.9-003: The system shall perform content-based filtering.
REQ-4.9-004: The system shall perform collaborative filtering.
REQ-4.9-005: The system shall perform context-based filtering.
REQ-4.9-006: The system shall calculate a score for each restaurant.
REQ-4.9-007: The system shall sort recommendation results based on scores.

REQ-4.9-008: The system shall ensure diversity in recommendation results.
REQ-4.9-009: The system shall prevent repeated exposure of the same restaurant.
REQ-4.9-010: The system shall provide explanation information along with recommendations.
REQ-4.9-011: The system shall display a message when recommendations are unavailable due to insufficient data.

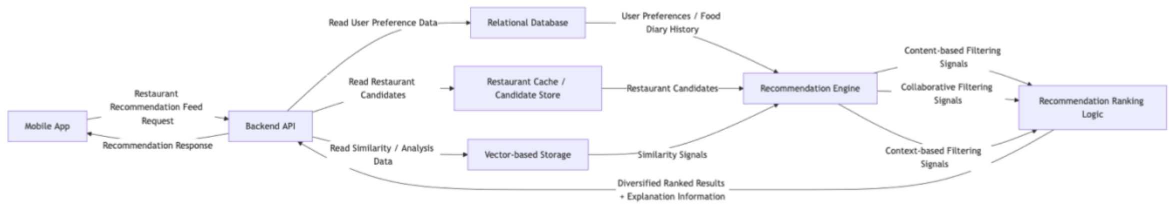

Figure 10. Restaurant Recommendation Data Flow Diagram

4.10 Settings and System Management

4.10.1 Description and Priority

This feature manages overall application settings including user preferences, notifications, terms, and customer support.

Priority: Medium
Benefit: 6 | Penalty: 6 | Cost: 4 | Risk: 4

4.10.2 Stimulus/Response Sequences

The user enters the settings screen.
The system displays the settings list.

The user changes notification settings.
The system saves the updated state.

The user views terms and policies.
The system displays them in a web view.

The user selects the contact option.
The system redirects to an external page.

4.10.3 Functional Requirements

REQ-4.10-001: The system shall provide notification ON/OFF functionality.
REQ-4.10-002: The system shall save setting changes immediately.
REQ-4.10-003: The system shall provide terms and policies via an in-app web view.
REQ-4.10-004: The system shall redirect users to a contact page.
REQ-4.10-005: The system shall display application version information.
REQ-4.10-006: The system shall display an error message in case of network failure.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

REQ-PERF-001: The system shall render the first 10 restaurant items within 3 seconds when entering the exploration screen.
REQ-PERF-002: All API calls shall have a timeout of 5 seconds or less.
REQ-PERF-003: Additional data shall load within 1 second during infinite scrolling.
REQ-PERF-004: The system shall support caching and progressive loading for images.
REQ-PERF-005: User interactions shall be reflected in the UI within 100 ms.

REQ-PERF-006: Camera initialization shall complete within 1.5 seconds.
REQ-PERF-007: Photo upload shall complete within 5 seconds under normal network conditions.
REQ-PERF-008: Diary saving shall complete within 3 seconds.
REQ-PERF-009: Diary list retrieval shall complete within 2 seconds.

REQ-PERF-010: Restaurant search response time shall be under 200 ms (database-level).
REQ-PERF-011: Searches including external APIs shall respond within 1 second.
REQ-PERF-012: Recommendation generation shall complete within 3 seconds.
REQ-PERF-013: Taste analysis shall complete within 30 seconds asynchronously.
REQ-PERF-014: The system shall support at least 100 concurrent users.

5.2 Safety Requirements

REQ-SAFE-001: Account deletion shall require a two-step confirmation process.
REQ-SAFE-002: Account deletion shall support recovery within a 30-day grace period.

REQ-SAFE-003: Diary entries shall be temporarily stored locally in case of network failure.
REQ-SAFE-004: The system shall automatically synchronize data when the network is restored.
REQ-SAFE-005: The system shall notify users if temporary data exists.

REQ-SAFE-006: The system shall preserve original image data if upload fails.
REQ-SAFE-007: The system shall prompt users before exiting with unsaved input.

REQ-SAFE-008: The system shall provide default recommendations in case of recommendation failure.
REQ-SAFE-009: The system shall preserve existing data if profile generation fails.

5.3 Security Requirements

REQ-SEC-001: All communication shall use HTTPS (TLS 1.2 or higher).
REQ-SEC-002: Authentication shall be performed using JWT.
REQ-SEC-003: Authentication tokens shall be stored securely.
REQ-SEC-004: User data shall only be accessible by the owner.
REQ-SEC-005: The database shall implement Row-Level Security (RLS).
REQ-SEC-006: External API keys shall be managed via server environment variables.
REQ-SEC-007: Sensitive information shall not be hardcoded.
REQ-SEC-008: User personal data shall not be transmitted to external services.
REQ-SEC-009: Image access shall be restricted via authentication.

5.4 Software Quality Attributes

REQ-QA-001: The system shall comply with WCAG 2.1 AA accessibility standards.
REQ-QA-002: The system shall support screen readers.
REQ-QA-003: Layout shall be maintained with up to 200% font scaling.
REQ-QA-004: Core functions shall be completed within three steps.
REQ-QA-005: All errors shall provide user-friendly messages.
REQ-QA-006: Empty states shall include guidance messages and action prompts.
REQ-QA-007: The system shall use cached data during network failure.
REQ-QA-008: The system shall automatically recover from data inconsistencies.
REQ-QA-009: The system shall use reusable component structures.
REQ-QA-010: Routing structure shall support maintainability.
REQ-QA-011: Recommendation algorithms shall be configurable.
REQ-QA-012: Analysis and recommendation processes shall be traceable via logs.

5.5 Business Rules

REQ-BIZ-001: User food diary data shall not be publicly disclosed.
REQ-BIZ-002: Recommendation functionality shall be enabled only when sufficient data exists.
REQ-BIZ-003: Rating-based recommendations shall be provided only under specific conditions.
REQ-BIZ-004: Duplicate bookmarks for the same restaurant shall not be allowed.
REQ-BIZ-005: Duplicate recommendations for the same restaurant shall be limited.
REQ-BIZ-006: Future-dated diary entries shall not be allowed.
REQ-BIZ-007: Restaurant data shall not be deleted.
REQ-BIZ-008: Recommendation scores shall not be exposed to users.

REQ-BIZ-009: User data used for recommendations shall not be disclosed.
REQ-BIZ-010: The map service shall be fixed to Kakao Map.

---

6. Other Requirements

Appendix A: Glossary

General Terms

SRS (Software Requirements Specification)
A document that defines the functions and constraints that a system must satisfy.

SnapPlate
A mobile application that records food experiences and provides personalized restaurant recommendations.

UI/UX Terms

Skeleton Screen
A UI pattern that displays placeholder layouts during content loading to reduce perceived latency.

Optimistic Update
A technique where the client updates the UI before receiving a server response and rolls back if necessary.

Bottom Sheet
A UI component that appears from the bottom of the screen.

Empty State
A screen displayed when no data is available, providing guidance to the user.

Haptic Feedback
A physical feedback mechanism such as vibration in response to user interaction.

System and Technical Terms

Deep Link
A method of navigating directly to a specific screen within an application.

Fallback
A backup mechanism used when a primary method fails.

Pre-fetch
A technique of loading data in advance before user requests.

Debouncing
A technique that processes only the last event in a sequence of rapid inputs.

Exponential Backoff
A retry strategy that increases delay intervals exponentially.

Cursor-based Pagination
A method of pagination using a cursor instead of offsets.

JWT (JSON Web Token)
A token-based authentication standard.

WCAG (Web Content Accessibility Guidelines)
A standard guideline for accessibility.

FCP (First Contentful Paint)
A performance metric indicating when the first content is rendered.

Algorithm Terms

Collaborative Filtering
A recommendation method based on similarities between users.

Content-Based Filtering
A recommendation method based on item attributes.

Appendix B: Taste Analysis Flow

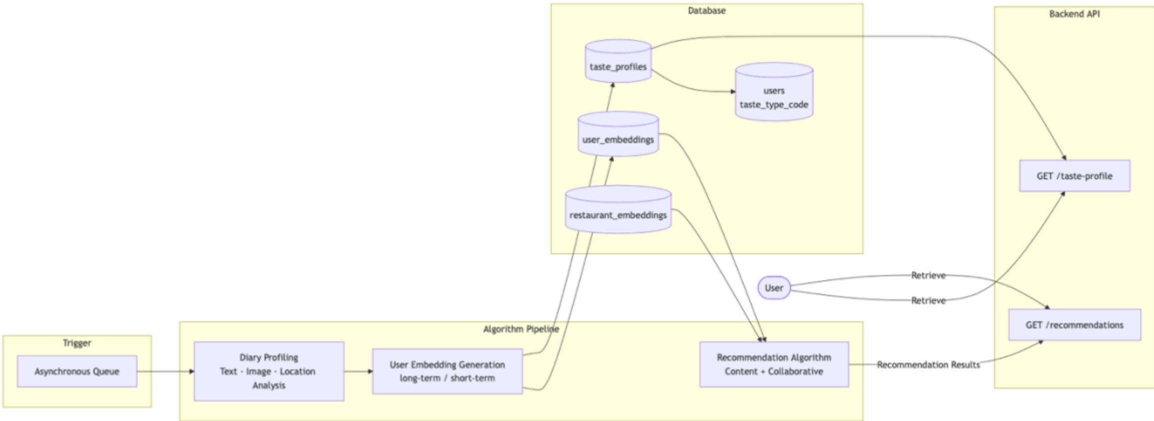

Figure 11. Taste Analysis Flow